

One of the readers (Krishna Kumar) had suggested reusing the dynamically allocated nodes using node pools. When a node is deleted by a thread, instead of freeing the memory, the node is returned to a per-thread free node list maintained by the application. The next time a node is requested by a thread, a node is returned from the thread's free node list, if available, instead of having to allocate memory dynamically. This will avoid frequent dynamic memory allocations and de-allocations. While this solution does reduce the number of mallocs/frees, it has its own disadvantages. It can result in memory remaining available in the free node list of some of the threads, while certain other threads run out of memory. What are the other issues that are caused by maintaining a free node pool?

There are special memory allocators designed for multi-threaded application, which can reduce the locking overhead due to dynamic memory allocation routines. Hoard is a popular dynamic memory allocator that exhibits good scalability in large scale multithreaded applications. Hoard is available from <http://prisms.cs.umass.edu/emery/index.php?page=download-hoard>. Another popular dynamic memory allocator is TCMalloc from Google, available from <http://goog-perftools.sourceforge.net/doc/tcmalloc.html>. An alternative approach to avoiding the lock contention issues on dynamic memory allocator data structures is to develop a lock-free dynamic memory allocator.

Atomicity violations

Our 12th question in last month's column was on atomicity violation. Consider the following code snippet taken from a MySQL source file *ha_innodb.cc*.

```
//Thread 1
S1: if (thd_proc_info)
{
    S2: fputs(thd_proc_info, _);
}
```

```
//Thread 2
S3: thd_proc_info=NULL;
```

Thread 1 and Thread 2 can execute concurrently. Consider that Thread 1 runs and executes S1. The scheduler schedules

out 'Thread 1' and now 'Thread 2' executes S3. 'Thread 1' runs again and executes S2. The question was about the wrong assumption made by the programmer in the above code snippet.

The programmer mistakenly assumes that the variable 'thd_proc_info' will not change between the point of checking to the time of use. However, in a multiprocessor environment, this assumption is not correct. Between the point of checking if the variable 'thd_proc_info' is 'non-null', to the point of using 'thd_proc_info', another thread can set the value of 'thd_proc_info' to NULL.

This month's takeaway question

While this bug is easier to see, consider the following code snippet:

```
volatile int X;
volatile int Y;
void foo()
{
    int result = X % Y;
    printf("%d is result\n", result);
}
```

On a certain computer system, the value of the result was greater than Y. Can you explain how/why that could happen?

Please do send me your answers to this month's questions. Also, if you are preparing for computer science programming interviews, you may find it useful to visit my programming interviews discussion group: 'Computer Science Interview Training (India)' on LinkedIn (www.linkedin.com). If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me along with your solutions and feedback at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and here's wishing you the very best! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.

A Computer Magazine that does not talk about Music, Games, Photo albums—Only Business

Read Benefit: A Business Minded Computer Magazine



www.benefitmag.com